

Guide de déploiement - DataVision AI v2.81.2

1. Principes

La règle principale est : **une release = un dossier de code propre**. Ne jamais extraire une nouvelle archive par-dessus un ancien dossier de projet.

Les fichiers hérités d'anciennes versions peuvent provoquer des échecs de repository hygiene et, plus grave, réintroduire des configurations obsolètes.

2. Prérequis Docker Compose

- Windows 10/11 ou Linux 64 bits ;
- Docker Desktop/Engine récent ;
- Docker Compose v2 ;
- accès au registre Docker et aux dépôts de paquets pendant le build ;
- mémoire/CPU suffisants pour PostgreSQL, Redis, API, worker, web et sandbox ;
- stockage persistant pour les volumes de données.

3. Extraction propre

Exemple Windows :

1. créer D:\Bureau\Project\datavision-v2.81.2 ;
2. extraire l'archive complète dans ce dossier ;
3. ne pas copier les nouveaux fichiers sur un ancien répertoire.

Si un ancien dossier a déjà été réutilisé :

```
powershell -ExecutionPolicy Bypass -File .\scripts\cleanup-legacy-root.ps1
```

Puis :

```
python scripts/repository_hygiene.py --root . --check
```

4. Configuration initiale

Copier .env.example vers .env, puis remplacer les secrets d'exemple.

Paramètres critiques :

```
APP_ENV=production          # uniquement en production réelle
AUTH_MODE=required
AUTH_SECRET=<secret aléatoire fort>
FIRST_RUN_SETUP_MODE=local
PASSWORD_MIN_LENGTH=8
DEMO_ACCOUNT_ENABLED=false  # obligatoire en production
```

CORS_ORIGINS=https://votre-domaine
API_DOCS_ENABLED=false

Configurer ensuite KMS/Vault, OIDC, antivirus, SMTP/connecteurs et autres services uniquement selon le profil utilisé.

5. Authentification

AUTH_MODE=required est le mode normal. Le mode local_dev est réservé au développement hors production.

Sur une installation locale, le premier compte propriétaire est créé depuis l'assistant de première configuration sans secret à saisir. DataVision émet en interne une session d'initialisation à durée courte, stockée dans un cookie HttpOnly, puis la rend inutilisable dès que le premier propriétaire existe.

Sur un serveur distant/production, le navigateur ne peut pas réclamer le premier compte. L'administrateur du serveur crée le propriétaire avec :

```
docker compose exec api python -m app.ops.bootstrap_owner --email admin@example.com
```

Le mot de passe est demandé de manière interactive et n'apparaît pas dans l'historique de commande.

Compte de démonstration

En développement local, .env.example active un compte de test : demo@datavision.local / DataVision8!. Il est créé dans une organisation et un workspace isolés avec le rôle admin. Les variables DEMO_ACCOUNT_* permettent de modifier ces valeurs.

Le runtime n'autorise ce compte que pour APP_ENV=development ou APP_ENV=test. En production, il est forcé hors service ; le doctor de configuration signale également DEMO_ACCOUNT_ENABLED=true comme configuration de production invalide.

6. Installation Windows recommandée

```
.\preflight-windows.ps1  
.\start-datavision.ps1
```

Le script d'installation génère les secrets nécessaires lorsque prévu. Conservez les valeurs sensibles hors des tickets, emails et captures d'écran.

7. Démarrage manuel Docker

```
docker compose down  
docker compose up --build
```

En cas de conflit de dépendances Python, vérifiez d'abord :

```
python scripts\dependency_compatibility.py --root . --check
```

La v2.81 corrige le conflit identifié entre boto3==1.40.40 et redshift-connector==2.1.16 en utilisant un pin compatible boto3==1.42.22.

8. Validation après démarrage

Vérifier :

- /health/live ;
- /health/startup ;
- /health/ready ;
- accès au frontend ;
- écran d'authentification ;
- première configuration locale ou login ;
- ouverture d'un workspace ;
- import d'un petit dataset ;
- exécution d'une analyse simple.

9. Reverse proxy et TLS

En production :

- HTTPS obligatoire ;
- certificat valide et renouvellement automatisé ;
- redirection HTTP->HTTPS ;
- taille des requêtes limitée ;
- timeouts adaptés ;
- headers proxy conservant correctement X-Forwarded-Proto ;
- API et services internes non exposés directement si non nécessaire.

10. CORS

N'utilisez pas * en production. Déclarez uniquement les origines réelles du frontend.

11. Docs API

Swagger/OpenAPI est désactivé par défaut. Ne l'activez en production que si nécessaire, sur une surface protégée et après revue de sécurité.

12. Kubernetes

Avant déploiement :

1. rendre le chart Helm ;
2. exécuter Conftest ;
3. vérifier non-root ;
4. allowPrivilegeEscalation=false ;
5. seccompProfile=RuntimeDefault ;
6. capabilities.drop: [ALL] ;
7. requests/limits ;
8. secrets fournis via un mécanisme sécurisé.

13. Upgrade

Pour Windows, utiliser upgrade-windows.ps1. La procédure doit :

1. vérifier la version source ;
2. créer/valider une sauvegarde ;
3. arrêter proprement ;
4. utiliser le nouveau dossier de release ;
5. appliquer les migrations ;
6. vérifier readiness ;
7. exécuter smoke tests ;
8. conserver les volumes tant que le rollback n'est pas abandonné.

14. Rollback

Un rollback doit utiliser une sauvegarde compatible et vérifiée. Ne supprimez jamais les volumes par réflexe. Documentez la cause, la version source/cible et les preuves de restauration.

15. Checklist de mise en production

- ☐ Repository hygiene OK
- ☐ Dependency compatibility OK
- ☐ pip check OK dans l'image construite
- ☐ frontend typecheck/build OK
- ☐ tests backend OK
- ☐ pip-audit/npm audit sans High/Critical non accepté
- ☐ Trivy image/filesystem OK
- ☐ Conftest OK
- ☐ secrets réels hors Git
- ☐ TLS validé
- ☐ CORS strict
- ☐ AUTH_MODE=required
- ☐ DEMO_ACCOUNT_ENABLED=false
- ☐ première configuration locale désactivée après création du premier owner
- ☐ KMS/Vault validé
- ☐ backup/restore drill validé
- ☐ pentest externe traité
- ☐ UAT signée